

第5讲 MATLAB数据分析与应用

(Chapter 5 MATLAB Data Analysis and Application)

5.1 数据统计处理 (Data Statistical Processing)

5.2 数据插值 (Data Interpolation)

5.3 曲线拟合 (Curve Fitting)

5.4 多项式计算 (Polynomial Calculation)

5.1 数据统计处理

(Data Statistical Processing)

5.1.1 最大值和最小值 (Max and Min)

MATLAB提供的求数据序列的最大值和最小值的函数分别为**max**和**min**，两个函数的调用格式和操作过程类似。

1. 求向量的最大值和最小值

求一个向量X的最大值的函数有两种调用格式，分别是：

- (1) **y=max(X)**：返回向量X的最大值存入y，如果X中包含复数元素，则按模取最大值。

(2) **[y,I]=max(X)**: 返回向量X的最大值存入y，最大值的序号存入I，如果X中包含复数元素，则按模取最大值。

求向量X的最小值的函数是**min(X)**，用法和**max(X)**完全相同。

Example 5-1 求向量x的最大值。

命令如下：

```
x=[-43,72,9,16,23,47];
```

```
y=max(x)           %求向量x中的最大值
```

```
[y,I]=max(x)       %求向量x中的最大值及其该元素  
                    的位置
```

2. 求矩阵的最大值和最小值

求矩阵A的最大值的函数有3种调用格式，分别是：

- (1) **max(A)**: 返回一个行向量，向量的第i个元素是矩阵A的第i列上的最大值。
- (2) **[Y,U]=max(A)**: 返回行向量Y和U，Y向量记录A的每列的最大值，U向量记录每列最大值的行号。
- (3) **max(A,[],dim)**: dim取1或2。dim取1时，该函数和max(A)完全相同；dim取2时，该函数返回一个列向量，其第i个元素是A矩阵的第i行上的最大值。

求最小值的函数是**min**，其用法和**max**完全相同。

Example 5-2 分别求下列 4×3 矩阵A中各列和各行元素中的最大值，并求整个矩阵的最大值和最小值。

$$A = \begin{pmatrix} 13 & -56 & 78 \\ 25 & 63 & -235 \\ 78 & 25 & 563 \\ 1 & 0 & -1 \end{pmatrix}$$

程序如下：

```
A=[13,-56,78;25,63,-235;78,25,563;1,0,-1];
```

```
Y=max(A);Z=max(A,[],2);y=max(Y);
```

```
z=min(min(A));
```

3. 两个向量或矩阵对应元素的比较

函数max和min还能对两个同型的向量或矩阵进行比较，调用格式为：

- (1) **U=max(A,B)**: A,B是两个同型的向量或矩阵，结果U是与A,B同型的向量或矩阵，U的每个元素等于A,B对应元素的较大者。
- (2) **U=max(A,n)**: n是一个标量，结果U是与A同型的向量或矩阵，U的每个元素等于A对应元素和n中的较大者。

min函数的用法和max完全相同。

Example 5-3 求两个 2×3 矩阵x, y所有同一位置上的较大元素构成的新矩阵p。

操作如下：

$x=[4,5,6;1,4,8]; y=[1,7,5;4,5,7]; p=\max(x,y)$

5.1.2 求和与求积 (Summation and Product)

数据序列求和与求积的函数是`sum`和`prod`，其使用方法类似。设 X 是一个向量， A 是一个矩阵，函数的调用格式为：

`sum(X)`：返回向量 X 各元素的和。

`prod(X)`：返回向量 X 各元素的乘积。

`sum(A)`：返回一个行向量，其第 i 个元素是 A 的第 i 列的元素和。

`prod(A)`：返回一个行向量，其第 i 个元素是 A 的第 i 列的元素乘积。

sum(A,dim): 当dim为1时, 该函数等同于sum(A);
当dim为2时, 返回一个列向量, 其第i个元素是A
的第i行的各元素之和。

prod(A,dim): 当dim为1时, 该函数等同于prod(A);
当dim为2时, 返回一个列向量, 其第i个元素是A
的第i行的各元素乘积。

Example 5-4 求矩阵A的每行元素的乘积和全部元素的乘积。

A=[1,2,3,4;5,6,7,8;9,10,11,12];

S=prod(A,2)

prod(S)

5.1.3 平均值和中值 (Mean and Median)

求数据序列平均值的函数是**mean**，求数据序列中值的函数是**median**。两个函数的调用格式为：

mean(X): 返回向量X的算术平均值。

median(X): 返回向量X的中值。

mean(A): 返回一个行向量，其第i个元素是A的第i列的算术平均值。

median(A): 返回一个行向量，其第i个元素是A的第i列的中值。

mean(A,dim): 当dim为1时，该函数等同于**mean(A)**；当dim为2时，返回一个列向量，其第i个元素是A的第i行的算术平均值。

median(A,dim): 当dim为1时，该函数等同于**median(A)**；当dim为2时，返回一个列向量，其第i个元素是A的第i行的中值。

5.1.4 累加和与累乘积 (Cumulative Sum and Cumulative Product)

在MATLAB中，使用**cumsum**和**cumprod**函数能方便地求得向量和矩阵元素的累加和与累乘积向量，函数的调用格式为：

cumsum(X)：返回向量X累加和向量。

cumsum(A)：返回一个矩阵，其第i列是A的第i列的累加和向量。

cumsum(A,dim)：当dim为1时，该函数等同于**cumsum(A)**；当dim为2时，返回一个矩阵，其第i行是A的第i行的累加和向量。

cumprod(X): 返回向量X累乘积向量。

cumprod(A): 返回一个矩阵，其第i列是A的第i列的累乘积向量。

cumprod(A,dim): 当dim为1时，该函数等同于
cumprod(A); 当dim为2时，返回一个向量，其第i行是A的第i行的累乘积向量。

Example 5-5 求向量y的平均值和中值。

y=[9,-2,5,6,7,12]

m=mean(y)

median(y)

Example 5-6 求向量X=(1!,2!,3!, ...,10!)。

X=cumprod(1:10)

5.1.5 标准方差与相关系数 (Standard Deviation and Correlation Coefficient)

1. 求标准方差 (Standard Deviation)

在MATLAB中，提供了计算数据序列的标准方差的函数**std**。对于向量X，std(X)返回一个标准方差。对于矩阵A，std(A)返回一个行向量，它的各个元素便是矩阵A各列或各行的标准方差。std函数的一般调用格式为：**Y=std(A,flag,dim)** 其中dim取1或2。当dim=1时，求各列元素的标准方差；当dim=2时，则求各行元素的标准方差。flag取0或1，当flag=0时，按S₁所列公式计算标准方差，当flag=1时，按S₂所列公式计算标准方差。缺省flag=0，dim=1。

$$S_1 = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}, S_2 = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2},$$

Example 5-7 对二维矩阵 x ，从不同维方向求出其标准方差。

命令如下：

```
x=[4,5,6;1,4,8];y1=std(x,0,1)
```

```
y2=std(x,1,1)
```

```
y3=std(x,0,2)
```

```
y4=std(x,1,2)
```

2. 相关系数 (Correlation Coefficient)

MATLAB提供了**corrcoef**函数，可以求出数据的相关系数矩阵。corrcoef函数的调用格式为：

corrcoef(X)：返回从矩阵 X 形成的一个相关系数矩阵。此相关系数矩阵的大小与矩阵 X 一样。它把矩阵 X 的每列作为一个变量，然后求它们的相关系数。

corrcoef(X,Y)：在这里， X,Y 是向量，它们与**corrcoef([X,Y])**的作用一样。

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}}$$

Example 5-8 生成满足正态分布的 10000×5 随机矩阵，然后求各列元素的均值和标准方差，再求这5列随机数据的相关系数矩阵。

命令如下：

X=randn(10000,5);

M=mean(X)

D=std(X)

R=corrcoef(X)

5.1.6 排序 (Sort)

MATLAB中对向量X是排序函数是**sort(X)**，函数返回一个对X中的元素按升序排列的新向量。

sort函数也可以对矩阵A的各列或各行重新排序，其调用格式为：

[Y,I]=sort(A,dim,mode)

其中 **dim** 指明对 A 的列还是行进行排序。若 **dim=1**，则按列排；若 **dim=2**，则按行排，**dim**默认取1。Y是排序后的矩阵，而I记录Y中的元素在A中位置。Mode指明按升序还是降序排列，'ascend'为升序，'descend'为降序，mode默认取'ascend'。

$$A = \begin{pmatrix} 1 & -8 & 5 \\ 4 & 12 & 6 \\ 13 & 7 & -13 \end{pmatrix}$$

Example 5-9 对下列矩阵做各种排序。

命令如下：

```
A=[1,-8,5;4,12,6;13,7,-13]
```

```
sort(A)           %对A的每列按升序排序
```

```
sort(A,2,'descend') %对A的每行按降序排序
```

```
[X,I]=sort(A)      %对A按列排序，并将每个元素所在的行号送矩阵I
```


5.2 数据插值 (Data Interpolation)

问题：工程测量和科学实验中所得到的数据通常都是离散的。若要得到这些离散点以外的其他点的数值，就需要根据这些已知数据进行插值。例如，测得 n 个数据点 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，这些数据点反映了一个函数关系 $y=f(x)$ ，然而并不知道 $f(x)$ 的解析式。

数值插值的任务就是根据上述条件构造一个函数 $y=g(x)$ ，使得在 $x_i (i=1, 2, \dots, n)$ ，有 $g(x_i)=f(x_i)$ ，且在两个相邻的采样点 $(x_i, x_{i+1}) (i=1, 2, \dots, n-1)$ 之间， $g(x)$ 光滑过渡。

5.2.1 一维数据插值 (One-dimensional data interpolation)

数据插值的任务是根据采集到的离散数据构造一个函数 $g(x)$ ，既与真实函数 $f(x)$ 接近，又有很好的性质。

在MATLAB中，实现这些插值的一维函数是 `interp1`，其调用格式为：

`Y1=interp1(X,Y,X1,'method')`

函数根据 X, Y 的值，计算函数在 $X1$ 处的值。 X, Y 是两个等长的已知向量，分别描述采样点和样本值， $X1$ 是一个向量或标量，描述欲插值的点， $Y1$ 是一个与 $X1$ 等长的插值结果。
`method` 是插值方法，允许的取值有 'linear'、'nearest'、'pchip'、'spline'。

注意： $X1$ 的取值范围不能超出 X 的给定范围，否则，会给出“NaN”错误提示，如 `linear` 和 `nearest` 方法，而其他的方法将执行外插值，所得结果往往与真值相差甚远。

linear: 线性插值（默认的插值方法）。是把与插值点靠近的两个数据点用直线连接，然后在直线上选取对应插值点的数据；

nearest: 最近的插值。根据已知插值点与已知数据点的远近程度进行插值。插值点优先选择较近的数据点进行插值操作；

pchip: 分段3次埃尔米特插值。采用分段三次多项式，除满足插值条件，还需要满足在若干点处的一阶导数也相等，从而提高了插值函数的光滑性。有一个专门的埃尔米特插值函数

pchip(X,Y,X1)，其功能与interp1(X,Y,X1, 'pchip')相同；

spline: 3次样条插值。在每个分段（子区间）内构造一个三次多项式，除满足插值条件，还要求在各节点处具有连续的一阶和二阶导数，从而保证节点处光滑。有一个专门的样条插值函数**spline(X,Y,X1)**，其功能与interp1(X,Y,X1, 'spline')相同。

Example 5-10 用不同的插值方法计算 $\sin x$ 在 $\pi/2$ 点的值。

命令如下：

```
X=0:0.2:pi;Y=sin(X); %给出 X、Y
>> interp1(X,Y,pi/2) %用默认方法(即线性插值方法)计算  $\sin(\pi/2)$ 
ans =
    0.9975
>> interp1(X,Y,pi/2, 'nearest') %用最近点插值方法计算  $\sin(\pi/2)$ 
ans =
    0.9996
>> interp1(X,Y,pi/2, 'linear') %用线性插值方法计算  $\sin(\pi/2)$ 
ans =
    0.9975
>> interp1(X,Y,pi/2,'pchip') %用3次埃尔米特插值方法计算  $\sin(\pi/2)$ 
ans =
    0.9992
>> interp1(x,Y,pi/2,'spline') %用3次样条插值方法计算  $\sin(\pi/2)$ 
ans =
    1.0000
```

Example 5-11 假设有一个汽车发动机在转速为2000r/min时，温度与时间的5个测量值如下表：

时间/s	0	1	2	3	4	5
温度/°C	0	20	60	68	77	110

其中温度的数据从20 °C变化到110 °C，试分别估计在t=2.5s和t=4.3s时的温度。

```
>>t=[0 1 2 3 4 5];      %输入时间
>>y=[0 20 60 68 77 110]; %输入温度
>>y1=interp1(t,y,2.5)    %计算要内插的数据点2.5的函数值
>>y1=interp1(t,y,[2.5,4.3]) %计算[]多个内插点
>>y1=interp1(t,y,2.5, 'pchip') %以3次方程式插值
>>y1=interp1(t,y,2.5, 'spline') %以样条插值
```

MATLAB中有一个专门的3次样条插值函数 **$Y1=spline(X,Y,X1)$** ，其功能及使用方法与函数 **$Y1=interp1(X,Y,X1, 'spline')$** 完全相同。

Example 5-13 某观测站测得某日 6:00 时至 18:00 时之间每隔两小时的室内外温度($^{\circ}\text{C}$)如下表所示, 用3次样条插值分别求得该日室内外 6:30 时至 17:30 时之间每隔两小时各点的近似温度($^{\circ}\text{C}$)。

时间h	6	8	10	12	14	16	18
室内温度t1	18.0	20.0	22.0	25.0	30.0	28.0	24.0
室外温度t2	15.0	19.0	24.0	28.0	34.0	32.0	30.0

设时间变量h为一行向量, 温度变量t为一个两列矩阵, 其中第1列存放室内温度, 第2列存放室外温度。

命令如下:

```
h=6:2:18;
```

```
t=[18, 20, 22, 25, 30, 28, 24; 15, 19, 24, 28, 34, 32, 30]';
```

```
X1=6.5:2:17.5
```

```
Y1=interp1(h,t,X1,'spline') %用3次样条插值计算
```

5.2.2 二维数据插值（Two-dimensional data interpolation）

在MATLAB中，提供了解决二维插值问题的函数**interp2**，其调用格式为：

Z1=interp2(X,Y,Z,X1,Y1,'method')

其中X,Y是两个向量，分别描述两个参数的采样点，Z是与参数采样点对应的函数值，X1,Y1是两个向量或标量，描述欲插值的点。Z1是根据相应的插值方法得到的插值结果。**method**的取值与一维插值函数相同，但二维插值不支持**pchip**方法。X、Y、Z也可以是矩阵形式。

注意：X1,Y1的取值范围不能超出X,Y的给定范围，否则，会给出“NaN”错误提示（如**linear**、**nearest**）或进行外插值（**spline**）。

Example 5-14 设 $z=x^2+y^2$ ，对 z 函数在 $[0,1]\times[0,2]$ 区域内进行插值。

```
x=0:0.1:1; y=0:0.2:2;
```

```
[X,Y]=meshgrid(x,y);      %产生自变量网格坐标
```

```
z=X.^2+Y.^2;              %求对应的函数值
```

```
interp2(x,y,z,0.5,0.5)    %在(0.5,0.5)点插值
```

```
ans =
```

```
    0.5100
```

```
interp2(x,y,z,[0.5,0.6],0.4) %在(0.5,0.4)和(0.6,0.4)点插值
```

```
ans =
```

```
    0.4100    0.5200
```

```
interp2(x,y,z,[0.5,0.6],[0.4,0.5]) %在(0.5,0.4)和(0.6,0.5)点插值
```

```
ans =
```

```
    0.4100    0.6200
```

```
%%---在(0.5,0.4)，(0.6,0.4)，(0.5,0.5)，(0.6,0.5)各点插值
```

```
interp2(x,y,z,[0.5,0.6]', [0.4,0.5])
```

```
ans =
```

```
    0.4100    0.5200
```

```
    0.5200    0.6200
```

```
%%---提高精度可用 'spline'
```

```
interp2(x,y,z,[0.5,0.6]',[0.4,0.5], 'spline')
```


Example 5-15 某实验对一根长10米的钢轨进行热源的温度传播测试。用x表示测量点0:2.5:10(米)，用h表示测量时间0:30:60(秒)，用T表示测试所得各点的温度(°C)。试用线性插值求出在一分钟内每隔20秒、钢轨每隔1米处的温度T1。

(钢轨各点温度测量值)

T \ x					
	0	2.5	5	7.5	10
h					
0	95	14	0	0	0
30	88	48	32	12	6
60	67	64	54	48	41

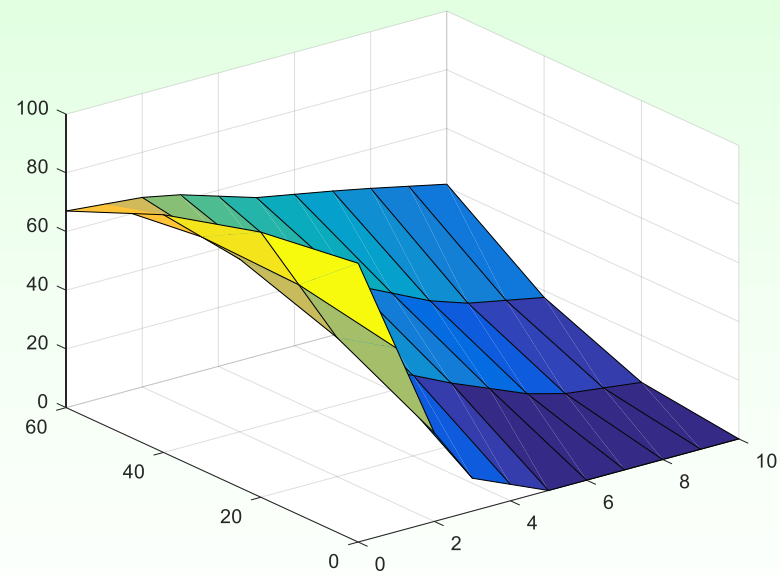
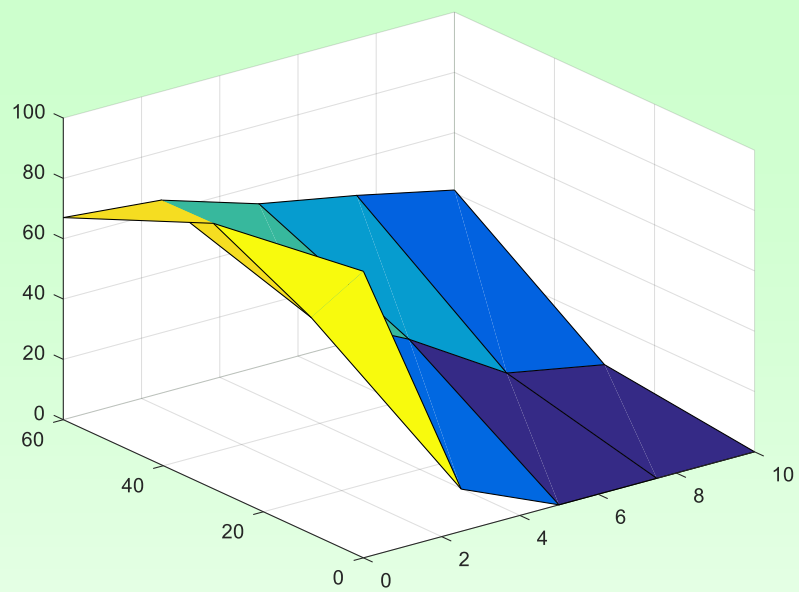
```
x=0:2.5:10;h=[0:30:60]';
```

```
T=[95,14,0,0,0;88,48,32,12,6;67,64,54,48,41];
```

```
x1=[0:10];h1=[0:20:60]';
```

```
T1=interp2(x,h,T,x1,h1);
```

```
surf(x,h,T); pause; surf(x1,h1,T1);
```



Experiment Content

Ex-1: 利用MATLAB提供的randn函数生成符合正态分布的 10×5 随机矩阵A，进行如下操作：

- (1) A各列元素的均值和标准差。
- (2) A的最大元素和最小元素。
- (3) 求A每行元素的和以及全部元素之和。
- (4) 分别对A的每列元素按升序、每行元素按降序。

Ex-2: 按下表用3次多项式方法插值计算1~100之间整数的平方。

N	1	4	9	16	25	36	49	64	81	100
$\sqrt{N}$	1	2	3	4	5	6	7	8	9	10

Experiment Content

Ex-3: 复现 Example 5-15, 并给出计算结果;

Ex-4: 已知下表的数据, 求当 $x_i=0.25$ 时的 y_i 的值, 并画出线性插值、三次样条插值、三次多项式插值的图形。

x	0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	0.3	0.5	1	1.4	1.6	1.9	0.6	0.4	0.8	1.5	2

思考：数值插值要求逼近函数在采样点与被逼近函数相等，但由于实验或测量中的误差，所获得的数据不一定准确。

为此，设想构造这样的函数 $y=g(x)$ 去逼近 $f(x)$ ，放弃在插值点两者完全相等的要求，使其在某种意义下最优。（可以考虑在最小二乘意义下）

5.3 曲线拟合 (Curve Fitting)

在MATLAB中，用**polyfit**函数来求得最小二乘拟合多项式的系数，再用**polyval**函数按所得的多项式计算所给出的点上的函数近似值。

polyfit函数的调用格式为：

$$[P,S]=\text{polyfit}(X,Y,m)$$

函数根据采样点 X 和采样点函数值 Y ，产生一个 m 次多项式 P 及其在采样点的误差向量 S 。其中 X,Y 是两个等长的向量， P 是一个长度为 $m+1$ 的向量， P 的元素为多项式系数。

polyval函数的功能是按多项式的系数计算 x 点多项式的值：

$$Y=\text{polyval}(P,x)$$

Example 5-16 用一个三次多项式在区间 $[0, 2\pi]$ 内逼近函数 $\sin(x)$ ，比较两曲线。

命令如下：

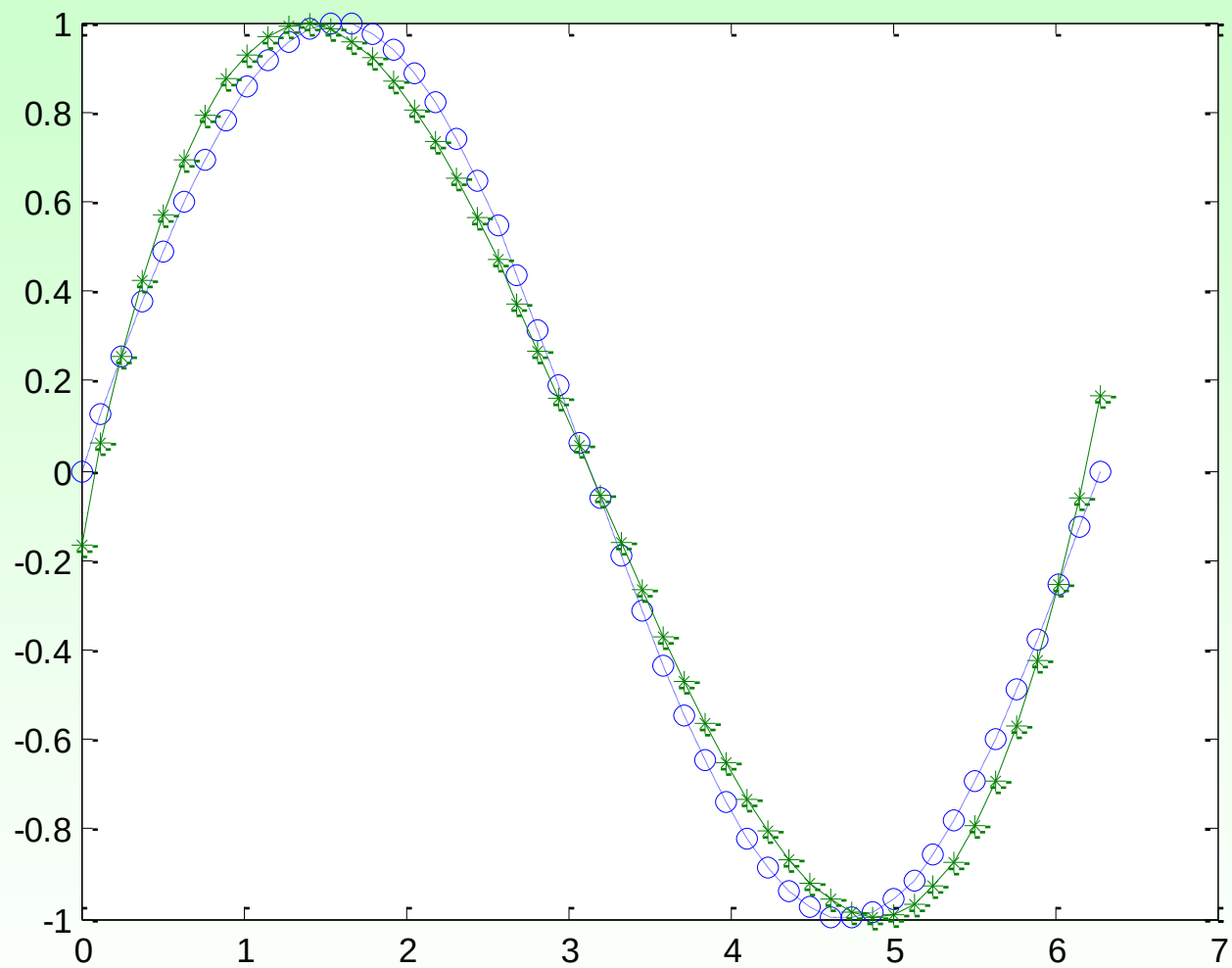
```
X=linspace(0,2*pi,50);
```

```
Y=sin(X);
```

```
[P,S]=polyfit(X,Y,3)      %得到3次多项式的系数和  
误差
```

```
y1=polyval(P,X);
```

```
plot(X,Y,':o',X,y1,'-*')
```



Example 5-17 测得某单分子化学反应速度数据如下表：

I	1	2	3	4	5	6	7	8
X_i	3	6	9	12	15	18	21	24
Y_i	57.6	41.9	31.0	22.7	16.6	12.2	8.9	6.5

其中 X_i 表示从实验开始算起的时间， Y_i 表示时刻反应物的量。根据化学反应速度的理论知道，选择的拟合模型应是指数函数 $y=ae^{bx}$ ，其中 a ， b 为待定参数。求拟合参数的最小二乘解。

解：拟合模型 $y=ae^{bx}$ 为非线性模型，两边取常用对数得到：
 $\log y = (b \log e)x + \log a$

令 $Y = \log y$, $B = 0.4343b$, $\log a = m$ ，则模型转化为：
 $Y = Bx + m$

首先利用 (x_i, Y_i) 进行一阶多项式拟合，然后根据 $B = 0.4343b$, $\log a = m$ 分别得到模型中的 a ， b 值。

```
x=[3 6 9 12 15 18 21 24];
```

```
y=[57.6 41.9 31.0 22.7 16.6 12.2 8.9 6.5];
```

```
Y=log10(y);
```

```
p1=polyfit(x,Y,1)
```

```
b=p1(1)/0.4343
```

```
a=10.^p1(2)
```

```
y1=polyval(p1,x)
```

最后得到拟合模型函数为: $y=78.57e^{-0.1037x}$

2、lsqcurvefit函数

用于非线性最小二乘拟合，也就是拟合一个非线性函数。

`x = lsqcurvefit(fun,x0,xdata,ydata)`

其中，fun是自定义的非线性函数句柄，x0是初始参数猜测值，xdata和ydata分别是数据点的横坐标和纵坐标向量，x是拟合得到的最优参数。

Example 5-18

```
fun = @(x,xdata) x(1)*exp(-x(2)*xdata); % 定义非线性函数
xdata = [3 6 9 12 15 18 21 24]; % 生成一些示例数据
ydata = [57.6 41.9 31.0 22.7 16.6 12.2 8.9 6.5];
x0 = [1, 0]; % 初始参数猜测值
x = lsqcurvefit(fun, x0, xdata, ydata)
```

% 生成拟合曲线的x值

```
x_fit = linspace(min(xdata), max(xdata), 100);
```

```
y_fit = fun(x, x_fit);
```

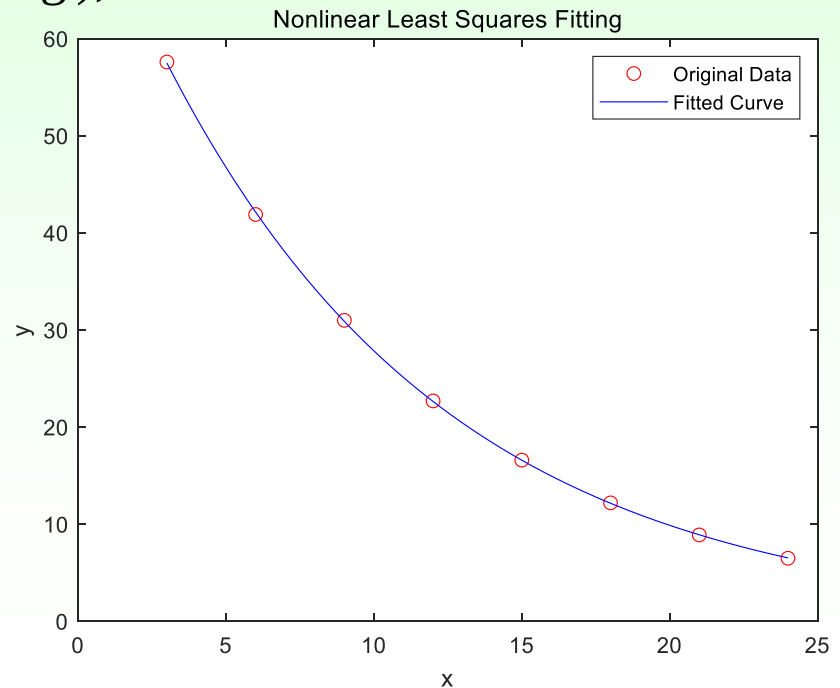
% 绘制原始数据和拟合曲线

```
plot(xdata, ydata, 'ro', x_fit, y_fit, 'b-');
```

```
legend('Original Data', 'Fitted Curve');
```

```
xlabel('x'); ylabel('y');
```

```
title('Nonlinear Least Squares Fitting');
```



3、fit函数

一个更高级的拟合工具，它能进行多项式、指数、对数等多种类型的拟合。

f = fit(x, y, fittype)

其中，x和y分别是数据点的横坐标和纵坐标向量（列向量），fittype是拟合类型，可以是‘poly1’（一次多项式）、‘poly2’（二次多项式）、‘exp1’（单指数函数）等。

Library Model Name	Description
'poly1'	Linear polynomial curve
'poly11'	Linear polynomial surface
'poly2'	Quadratic polynomial curve
'linearinterp'	Piecewise linear interpolation
'cubicinterp'	Piecewise cubic interpolation
'smoothingspline'	Smoothing spline (curve)
'lowess'	Local linear regression (surface)

Example 5-19

% 生成一些示例数据

```
x=[3 6 9 12 15 18 21 24];
```

```
y=[57.6 41.9 31.0 22.7 16.6 12.2 8.9 6.5];
```

% 进行一次多项式拟合

```
f = fit(x', y', 'exp1');
```

% 生成拟合曲线的x值

```
x_fit = linspace(min(x), max(x), 100);
```

```
y_fit = f(x_fit);
```

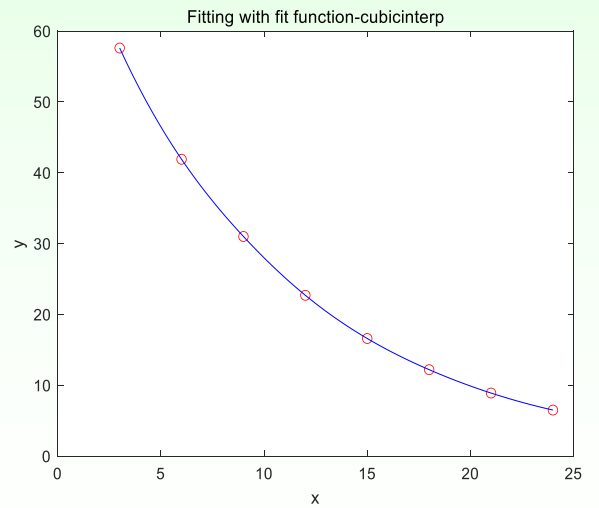
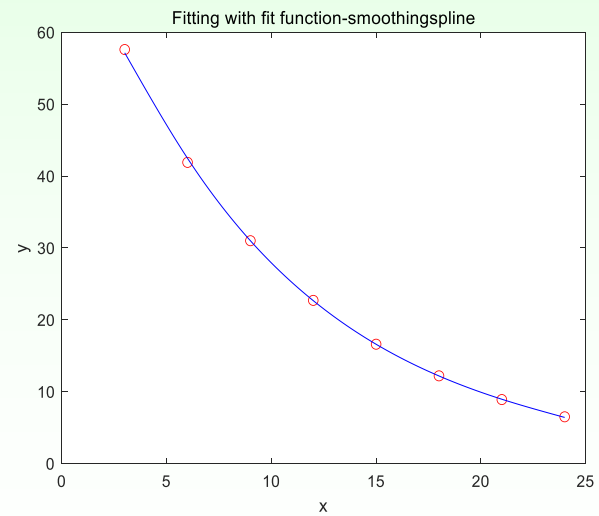
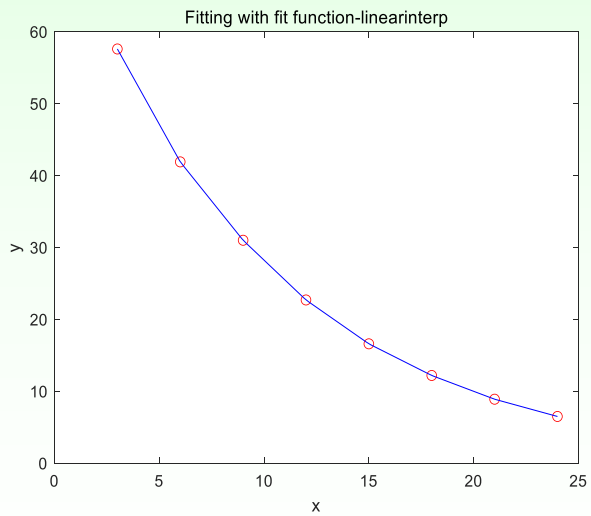
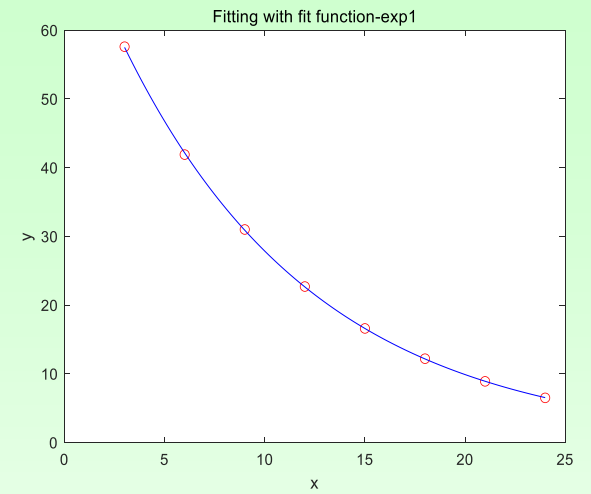
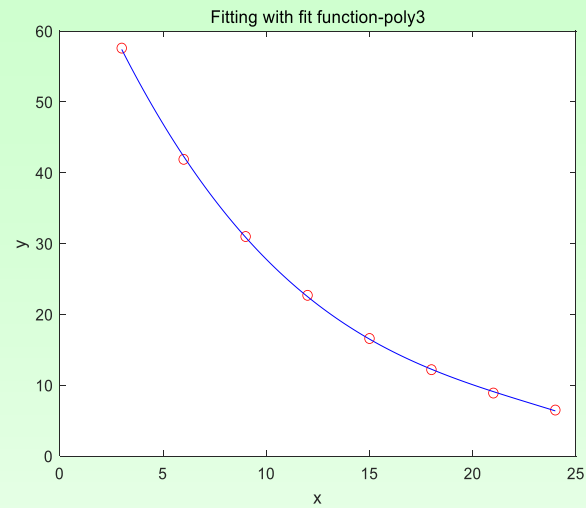
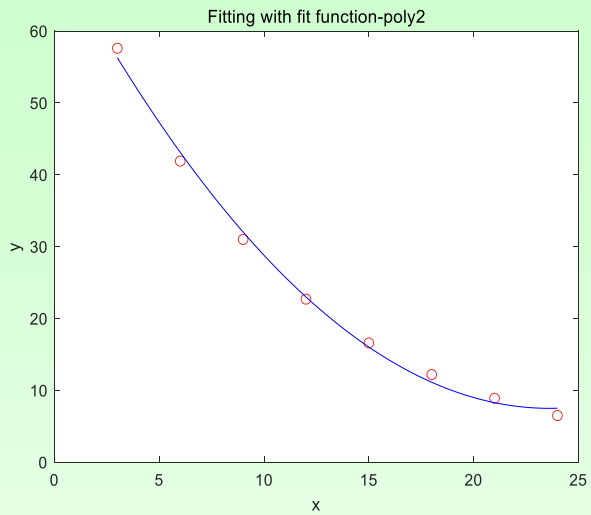
% 绘制原始数据和拟合曲线

```
plot(x, y, 'ro', x_fit, y_fit, 'b-');
```

```
xlabel('x');
```

```
ylabel('y');
```

```
title('Fitting with fit function');
```



5.4 多项式计算 (Polynomial Calculation)

5.4.1 多项式的四则运算 (Four Rules of Polynomial)

1. 多项式的加减运算 (Addition and Subtraction)

系数向量加减。

2. 多项式乘法运算 (Multiplication)

函数 **conv(P1,P2)** 用于求多项式 P1 和 P2 的乘积。这里，P1、P2 是两个多项式系数向量。

Example 5-16 求多项式 x^4+8x^3-10 与多项式 $2x^2-x+3$ 的乘积。

`a=[1,8,0,0,-10];b=[2,-1,3]; conv(a,b)`

3. 多项式除法 (Division)

函数 $[Q,r]=\text{deconv}(P1,P2)$ 用于对多项式 $P1$ 和 $P2$ 作除法运算。其中 Q 返回多项式 $P1$ 除以 $P2$ 的商式， r 返回 $P1$ 除以 $P2$ 的余式。这里， Q 和 r 仍是多项式系数向量。

deconv 是 conv 的逆函数，即有

$$P1 = \text{conv}(P2, Q) + r。$$

Example 5-17 求多项式 x^4+8x^3-10 除以多项式 $2x^2-x+3$ 的结果。

$$[Q,r]=\text{deconv}(a,b)$$

5.4.2 多项式的导函数 (Derivatives)

对多项式求导数的函数是：

p=polyder(P): 求多项式P的导函数

p=polyder(P,Q): 求P Q的导函数

[p,q]=polyder(P,Q): 求P/Q的导函数，导函数的分子存入p，分母存入q。

上述函数中，参数P,Q是多项式的向量表示，结果p,q也是多项式的向量表示。

Example 5-18 求有理分式的导数。

$$f(x) = \frac{3x^5 + 5x^4 - 8x^2 + x - 5}{10x^{10} + 5x^9 + 6x^6 + 7x^3 - x^2 - 100}$$

命令如下:

P=[3,5,0,-8,1,-5];

Q=[10,5,0,0,6,0,0,7,-1,0,-100];

[p,q]=polyder(P,Q)

p =

Columns 1 through 8

-150	-360	-125	640	172	400	225	234
------	------	------	-----	-----	-----	-----	-----

Columns 9 through 15

-4	170	-1444	-2014	106	1590	-100
----	-----	-------	-------	-----	------	------

q =

Columns 1 through 8

100	100	25	0	120	60	0	140
-----	-----	----	---	-----	----	---	-----

Columns 9 through 16

86	-10	-2000	-916	-12	0	-1151	-14
----	-----	-------	------	-----	---	-------	-----

Columns 17 through 21

1	-1400	200	0	10000
---	-------	-----	---	-------

5.4.3 多项式的求值 (Polynomial Evaluation)

MATLAB提供了两种求多项式值的函数：**polyval**与**polyvalm**，它们的输入参数均为多项式系数向量P和自变量x。两者的区别在于前者是代数多项式求值，而后者是矩阵多项式求值。

1. 代数多项式求值 (Evaluation)

polyval函数用来求代数多项式的值，其调用格式为：

$$Y = \text{polyval}(P, x)$$

若 x 为一数值，则求多项式在该点的值；若 x 为向量或矩阵，则对向量或矩阵中的每个元素求其多项式的值。

Example 5-19 已知多项式 $x^4 + 8x^3 - 10$ ，分别取 $x=1.2$ 和一个 2×3 矩阵为自变量计算该多项式的值。

```
A=[1,8,0,0,-10];    %4次多项式系数
```

```
x=1.2;              %取自变量为一数值
```

```
y1=polyval(A,x)
```

```
x=[-1,1.2,-1.4;2,-1.8,1.6];    %给出一个矩阵x
```

```
y2=polyval(A,x)
```

```
A=[1,8,0,0,-10];  
x=1.2;  
y1=polyval(A,x)  
x=[-1,1.2,-1.4;2,-1.8,1.6];  
y2=polyval(A,x)
```

```
y1 =  
    5.8976
```

```
y2 =  
-17.0000    5.8976 -28.1104  
 70.0000 -46.1584  29.3216
```

2. 矩阵多项式求值

polyvalm函数用来求矩阵多项式的值，其调用格式与**polyval**相同，但含义不同。**polyvalm**函数要求x为方阵，它以方阵为自变量求多项式的值。设A为方阵，P代表多项式 x^3-5x^2+8 ，那么**polyvalm(P,A)**的含义是：

$$\underline{A*A*A-5*A*A+8*eye(size(A))}$$

而**polyval(P,A)**的含义是：

$$\underline{A.*A.*A-5*A.*A+8*ones(size(A))}$$

Example 5-20 仍以多项式 x^4+8x^3-10 为例，取一个 2×2 矩阵为自变量分别用polyval和polyvalm计算该多项式的值。

```
A=[1,8,0,0,-10];    %多项式系数
x=[-1,1.2;2,-1.8];    %给出一个矩阵x
y1=polyval(A,x)        %计算代数多项式的值
y2=polyvalm(A,x)        %计算矩阵多项式的值
```

```
y1 =
    -17.0000    5.8976
    70.0000   -46.1584
y2 =
   -60.5840    50.6496
    84.4160   -94.3504
```

5.4.4 多项式求根 (Polynomial Roots)

n 次多项式具有 n 个根，当然这些根可能是实根，也可能含有若干对共轭复根。MATLAB提供的**roots**函数用于求多项式的全部根，其调用格式为：

$x = \text{roots}(P)$

其中 P 为多项式的系数向量，求得的根赋给向量 x ，即 $x(1), x(2), \dots, x(n)$ 分别代表多项式的 n 个根。

Example 5-21 求多项式 x^4+8x^3-10 的根。

命令如下：

```
A=[1,8,0,0,-10];
```

```
x=roots(A)
```

若已知多项式的全部根，则可以用poly函数建立起该多项式，其调用格式为：

P=poly(x)

若x为具有n个元素的向量，则poly(x)建立以x为其根的多项式，且将该多项式的系数赋给向量P。

Example 5-22 已知 $f(x)=3x^5+4x^3-5x^2-7.2x+5$

(1) 计算 $f(x)=0$ 的全部根。

(2) 由方程 $f(x)=0$ 的根构造一个多项式 $g(x)$ ，并与 $f(x)$ 进行对比。

命令如下：

```
P=[3,0,4,-5,-7.2,5];
```

```
X=roots(P)           %求方程 $f(x)=0$ 的根
```

```
G=poly(X)            %求多项式 $g(x)$ 
```

```
G =
```

```
1.0000    0.0000    1.3333   -1.6667   -2.4000  
1.6667
```

Experiment Content

Ex-1: 按要求对指定函数进行插值和拟合：按表1用三次样条方法插值计算0~90°范围内整数点的正弦值和0~75°范围内整数点的正切值，然后用5次多项式拟合方法计算相同的函数值，并将两种计算结果进行比较

表1 特殊角的正弦值和正切值表

a(度)	0	15	30	45	60	75	90
sin(a)	0	0.2588	0.5000	0.7071	0.8660	0.9659	1.0000
tan(a)	0	0.2679	0.5774	1.0000	1.7320	3.7320	

Ex-2: 已知一组实验数据入表2

表2 一组实验数据

i	1	2	3	4	5
x_i	165	123	150	123	141
y_i	187	126	172	125	148

求它的线性拟合曲线。

Ex-3: 已知多项式

$$P_1(x) = 3x + 2, P_2(x) = 5x^2 - x + 2, P_3(x) = x^2 - 0.5$$

求:

(1) $P(x) = P_1(x)P_2(x)P_3(x)$;

(2) $P(x) = 0$ 的全部根。